

[Introduction](#)[Going to production](#)[Web APIs reference](#)[Extend with Go](#)[Extend with JavaScript](#)[JS Overview](#)[JS Event hooks](#)[JS Routing](#)[JS Database](#)[JS Record operations](#)[JS Collection operations](#)[JS Migrations](#)[JS Jobs scheduling](#)[JS Sending emails](#)[JS Rendering templates](#)[JS Console commands](#)[JS Sending HTTP requests](#)[JS Realtime messaging](#)[JS Filesystem](#)[JS Logging](#)

Extend with JavaScript - Console commands

You can register custom console commands using

`app.rootCmd.addCommand(cmd)`, where `cmd` is a [Command](#) instance.

Here is an example:

```
$app.rootCmd.addCommand(new Command({
  use: "hello",
  run: (cmd, args) => {
    console.log("Hello world!")
  },
}))
```

To run the command you can execute:

```
./pocketbase hello
```



Keep in mind that the console commands execute in their own separate app process and run independently from the main `serve` command (aka. hook and realtime events between different processes are not shared with one another).

[← Prev: Rendering templates](#)[Next: Sending HTTP requests →](#)

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation



[Introduction](#)[Going to production](#)[Web APIs reference](#)Extend with
GoExtend with
JavaScript[JS Overview](#)[JS Event hooks](#)[JS Routing](#)[JS Database](#)[JS Record operations](#)[JS Collection operations](#)[JS Migrations](#)[JS Jobs scheduling](#)[JS Sending emails](#)[JS Rendering templates](#)[JS Console commands](#)[JS Sending HTTP requests](#)[JS Realtime messaging](#)[JS Filesystem](#)[JS Logging](#)

Extend with JavaScript - Sending HTTP requests

- [Overview](#)
 - [multipart/form-data requests](#)
- [Limitations](#)

Overview

You can use the global `$http.send(config)` helper to send HTTP requests to external services.

This could be used for example to retrieve data from external data sources, to make custom requests to a payment provider API, etc.

Below is a list with all currently supported config options and their defaults.

```
// throws on timeout or network connectivity error
const res = $http.send({
  url:      "",
  method:   "GET",
  body:     "", // ex. JSON.stringify({"test": 123}) or new FormData()
  headers:  {}, // ex. {"content-type": "application/json"}
  timeout:  120, // in seconds
})
```

```
console.log(res.headers) // the response
                           headers (ex. res.headers['X-Custom'][0])
console.log(res.cookies) // the response
                           cookies (ex. res.cookies.sessionId.value)
console.log(res.statusCode) // the response HTTP
                             status code
console.log(res.body)       // the response body
                             as plain bytes array
console.log(res.json)       // the response body
                             as parsed json array or map
```

Here is an example that will enrich a single book record with some data based on its ISBN details from openlibrary.org.

```
onRecordCreateRequest((e) => {
  let isbn = e.record.get("isbn");

  // try to update with the published date from
  the openlibrary API
  try {
    const res = $http.send({
      url: "https://openlibrary.org/isbn/" +
        isbn + ".json",
      headers: {"content-type":
        "application/json"}
    })

    if (res.statusCode == 200) {
      e.record.set("published",
        res.json.publish_date)
    }
  } catch (err) {
    e.app.logger().error("Failed to retrieve
    book data", "error", err);
  }

  return e.next()
}, "books")
```

multipart/form-data requests

In order to send `multipart/form-data` requests (ex. uploading files) the request `body` must be a `FormData` instance.

PocketBase JSVM's `FormData` has the same APIs as its browser equivalent with the main difference that for file values instead of `Blob` it accepts `$filesystem.File`.

```
const formData = new FormData();

formData.append("title", "Hello world!")
formData.append("documents",
  $filesystem.fileFromBytes("doc1", "doc1.txt"))
formData.append("documents",
  $filesystem.fileFromBytes("doc2", "doc2.txt"))

const res = $http.send({
  url: "https://...",
  method: "POST",
  body: formData,
})

console.log(res.statusCode)
```

Limitations

As of now there is no support for streamed responses or server-sent events (SSE). The `$http.send` call blocks and returns the entire response body at once.

For this and other more advanced use cases you'll have to [extend PocketBase with Go](#).

[← Prev: Console commands](#)

[Next: Realtime messaging →](#)

[FAQ](#)

[JavaScript SDK](#)

[PocketBase](#)

[Discussions](#)

[Dart SDK](#)

[Documentation](#)



[Introduction](#)[Going to production](#)[Web APIs reference](#)[Extend with Go](#)[Extend with JavaScript](#)[JS Overview](#)[JS Event hooks](#)[JS Routing](#)[JS Database](#)[JS Record operations](#)[JS Collection operations](#)[JS Migrations](#)[JS Jobs scheduling](#)[JS Sending emails](#)[JS Rendering templates](#)[JS Console commands](#)[JS Sending HTTP requests](#)[JS Realtime messaging](#)[JS Filesystem](#)[JS Logging](#)

Extend with JavaScript - Realtime messaging

By default PocketBase sends realtime events only for Record create/update/delete operations (*and for the OAuth2 auth redirect*), but you are free to send custom realtime messages to the connected clients via the `$app.subscriptionsBroker()` instance.

`$app.subscriptionsBroker().clients()` returns all connected `subscriptions.Client` indexed by their unique connection id.

The current auth record associated with a client could be accessed through `client.get("auth")`



Note that a single authenticated user could have more than one active realtime connection (aka. multiple clients). This could happen for example when opening the same app in different tabs, browsers, devices, etc.

Below you can find a minimal code sample that sends a JSON payload to all clients subscribed to the "example" topic:

```
const message = new SubscriptionMessage({
  name: "example",
  data: JSON.stringify({ ... }),
});

// retrieve all clients (clients id indexed map)
const clients =
  $app.subscriptionsBroker().clients()

for (let clientId in clients) {
  if
  (clients[clientId].hasSubscription("example")) {
    clients[clientId].send(message)
  }
}
```

From the client-side, users can listen to the custom subscription topic by doing something like:

JavaScript

Dart

```
import PocketBase from 'pocketbase';

const pb = new
PocketBase('http://127.0.0.1:8090');

...

await pb.realtime.subscribe('example', (e) => {
  console.log(e)
})
```

← Prev: Sending HTTP requests

Next: Filesystem →

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation



Extend with JavaScript - Filesystem

PocketBase comes with a thin abstraction between the local filesystem and S3.

To configure which one will be used you can adjust the storage settings from *Dashboard > Settings > Files storage* section.

The filesystem abstraction can be accessed programmatically via the `$app.newFilesystem()` method.

Below are listed some of the most common operations but you can find more details in the `filesystem.System` interface.



Always make sure to call `close()` at the end for both the created filesystem instance and the retrieved file readers to prevent leaking resources.

- [Reading files](#)
- [Saving files](#)
- [Deleting files](#)

Reading files

To retrieve the file content of a single stored file you can use `getReader(key)`.

Note that file keys often contain a **prefix** (aka. the "path" to the file). For record files the full key is

`collectionId/recordId/filename`.

To retrieve multiple files matching a specific *prefix* you can use `list(prefix)`.

The below code shows a minimal example how to retrieve the content of a single record file as string.

```
let record = $app.findAuthRecordByEmail("users",  
  "test@example.com")  
  
// construct the full file key by concatenating
```



👉 Going to production

📖 Web APIs reference

Extend with
Go

Extend with
JavaScript

JS Overview

JS Event hooks

JS Routing

JS Database

JS Record operations

JS Collection operations

JS Migrations

JS Jobs scheduling

JS Sending emails

JS Rendering templates

JS Console commands

JS Sending HTTP requests

JS Realtime messaging

JS **Filesystem**

JS Logging

JS Types reference

```
let fsys, reader, content;

try {
  // initialize the filesystem
  fsys = $app.newFilesystem();

  // retrieve a file reader for the avatar key
  reader = fsys.getReader(avatarKey)

  // copy as plain string
  content = toString(reader)
} finally {
  reader?.close();
  fsys?.close();
}
```

Saving files

There are several methods to save (*aka. write/upload*) files depending on the available file content source:

- `upload(content, key)` ↗
- `uploadFile(file, key)` ↗
- `uploadMultipart(mfh, key)` ↗

Most users rarely will have to use the above methods directly because for collection records the file persistence is handled transparently when saving the record model (it will also perform size and MIME type validation based on the collection `file` field options). For example:

```
let record = $app.findRecordById("articles",
  "RECORD_ID")

// Other available File factories
// - $filesystem.fileFromBytes(content, name)
// - $filesystem.fileFromURL(url)
// - $filesystem.fileFromMultipart(mfh)
let file =
  $filesystem.fileFromPath("/local/path/to/file")

// set new file (can be single or array of File
// values)
// (if the record has an old file it is
// automatically deleted on successful save)
record.set("yourFileField", file)
```

```
$app.save(record)
```

Deleting files

Files can be deleted from the storage filesystem using

`delete(key)` [↗](#) .

Similar to the previous section, most users rarely will have to use the `delete` file method directly because for collection records the file deletion is handled transparently when removing the existing filename from the record model (this also ensures that the db entry referencing the file is also removed). For example:

```
let record = $app.findRecordById("articles",  
  "RECORD_ID")  
  
// if you want to "reset" a file field (aka.  
// deleting the associated single or multiple files)  
// you can set it to null  
record.set("yourFileField", null)  
  
// OR if you just want to remove individual  
// file(s) from a multiple file field you can use the  
// "-" modifier  
// (the value could be a single filename string or  
// slice of filename strings)  
record.set("yourFileField-",  
  "example_52iWbGinWd.txt")  
  
$app.save(record)
```

[← Prev: Realtime messaging](#)

[Next: Logging →](#)



Extend with JavaScript - Logging

`$app.logger()` could be used to write any logs into the database so that they can be later explored from the PocketBase *Dashboard > Logs* section.

For better performance and to minimize blocking on hot paths, logs are written with debounce and on batches:



- 3 seconds after the last debounced log write
- when the batch threshold is reached (currently 200)
- right before app termination to attempt saving everything from the existing logs queue

- Logger methods
 - debug(message, attrs...)
 - info(message, attrs...)
 - warn(message, attrs...)
 - error(message, attrs...)
 - with(attrs...)
 - withGroup(name)
- Logs settings
- Custom log queries
- Intercepting logs write

Logger methods

All standard `log.Logger` methods are available but below is a list with some of the most notable ones. Note that attributes are represented as key-value pair arguments.

debug(message, attrs...)

✔ Going to production

☰ Web APIs reference

Extend with
Go

Extend with
JavaScript

JS Overview

JS Event hooks

JS Routing

JS Database

JS Record operations

JS Collection operations

JS Migrations

JS Jobs scheduling

JS Sending emails

JS Rendering templates

JS Console commands

JS Sending HTTP requests

JS Realtime messaging

JS Filesystem

JS **Logging**

JS Types reference

```
$app.logger().debug(
  "Debug message with attributes!",
  "name", "John Doe",
  "id", 123,
)
```

info(message, attrs...)

```
$app.logger().info("Info message!")

$app.logger().info(
  "Info message with attributes!",
  "name", "John Doe",
  "id", 123,
)
```

warn(message, attrs...)

```
$app.logger().warn("Warning message!")

$app.logger().warn(
  "Warning message with attributes!",
  "name", "John Doe",
  "id", 123,
)
```

error(message, attrs...)

```
$app.logger().error("Error message!")

$app.logger().error(
  "Error message with attributes!",
  "id", 123,
  "error", err,
)
```

with(attrs...)

`with(attrs...)` creates a new local logger that will "inject" the specified attributes with each following log.

```
const l = $app.logger().with("total", 123)

// results in log with data {"total": 123}
```

```
l.info("message A")

// results in log with data {"total": 123, "name": "john"}
l.info("message B", "name", "john")
```

withGroup(name)

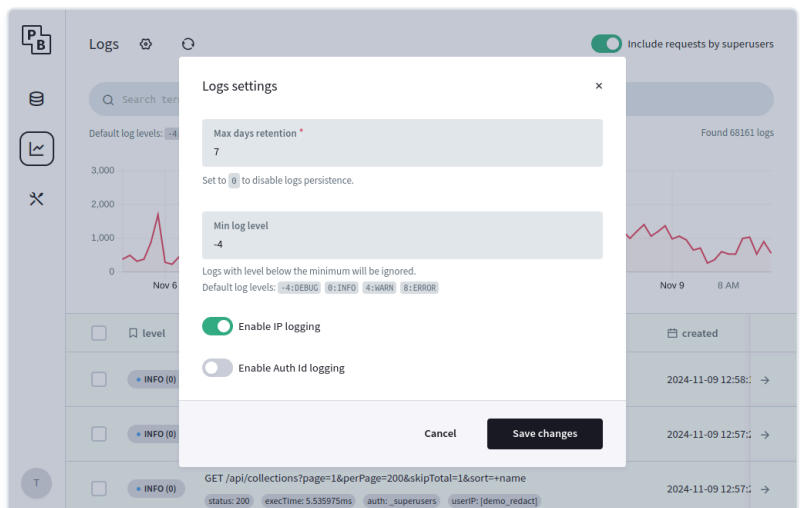
`withGroup(name)` creates a new local logger that wraps all logs attributes under the specified group name.

```
const l = $app.logger().withGroup("sub")

// results in log with data {"sub": { "total": 123 }}
l.info("message A", "total", 123)
```

Logs settings

You can control various log settings like logs retention period, minimal log level, request IP logging, etc. from the logs settings panel:



Custom log queries

The logs are usually meant to be filtered from the UI but if you want to programmatically retrieve and filter the stored logs you can make use of the `$app.logQuery()` query builder method. For example:

```
let logs = arrayOf(new DynamicModel({
  id: "",
```

```

    created: "",
    message: "",
    level: 0,
    data: {},
  })

// see https://pocketbase.io/docs/js-
// database/#query-builder
$app.logQuery().
  // target only debug and info logs
  andWhere($dbx.in("level", -4, 0)).
  // the data column is serialized json object
  // and could be anything
  andWhere($dbx.exp("json_extract(data,
    '$.type') = 'request'")).
  orderBy("created DESC").
  limit(100).
  all(logs)

```

Intercepting logs write

If you want to modify the log data before persisting in the database or to forward it to an external system, then you can listen for changes of the `_logs` table by attaching to the base model hooks. For example:

```

onModelCreate((e) => {
  // print log model fields
  console.log(e.model.id)
  console.log(e.model.created)
  console.log(e.model.level)
  console.log(e.model.message)
  console.log(e.model.data)

  e.next()
}, "_logs")

```

← Prev: Filesystem

Next: Types reference →

Discussions

Dart SDK



Documentation

[Introduction](#)[Going to production](#)[Web APIs reference](#)[API Records](#)[API Realtime](#)[API Files](#)[API Collections](#)[API Settings](#)[API Logs](#)[API Crons](#)[API Backups](#)[API Health](#)[Extend with Go](#)[Extend with JavaScript](#)[JS Overview](#)[JS Event hooks](#)[JS Routing](#)[JS Database](#)[JS Record operations](#)[JS Collection operations](#)[JS Migrations](#)[JS Jobs scheduling](#)[JS Sending emails](#)[JS Rendering templates](#)

Web APIs reference - API Realtime

The Realtime API is implemented via Server-Sent Events (SSE). Generally, it consists of 2 operations:

1. establish SSE connection
2. submit client subscriptions

SSE events are sent for **create**, **update** and **delete** record operations.

You could subscribe to a single record or to an entire collection.

When you subscribe to a **single record**, the collection's **ViewRule** will be used to determine whether the subscriber has access to receive the event message.

When you subscribe to an **entire collection**, the collection's **ListRule** will be used to determine whether the subscriber has access to receive the event message.

[Connect](#)[Set subscriptions](#)

All of this is seamlessly handled by the SDKs using just the `subscribe` and `unsubscribe` methods:

[JavaScript](#)[Dart](#)

```
import PocketBase from 'pocketbase';

const pb = new
PocketBase('http://127.0.0.1:8090');

...

// (Optionally) authenticate
```

```
await
pb.collection('users').authWithPassword('test@example.com', '1234567890');

// Subscribe to changes in any record in the collection
pb.collection('example').subscribe('*', function (e) {
  console.log(e.action);
  console.log(e.record);
}, { /* other options like expand, custom headers, etc. */ });

// Subscribe to changes only in the specified record
pb.collection('example').subscribe('RECORD_ID', function (e) {
  console.log(e.action);
  console.log(e.record);
}, { /* other options like expand, custom headers, etc. */ });

// Unsubscribe
pb.collection('example').unsubscribe('RECORD_ID');
// remove all 'RECORD_ID' subscriptions
pb.collection('example').unsubscribe('*'); // remove all '*' topic subscriptions
pb.collection('example').unsubscribe(); // remove all subscriptions in the collection
```

[← Prev: API Records](#)[Next: API Files →](#)[FAQ](#)[JavaScript SDK](#)[PocketBase](#)[Discussions](#)[Dart SDK](#)[Documentation](#)

[🏠 Introduction](#)[🚀 Going to production](#)[📖 Web APIs reference](#)[└ API Records](#)[└ API Realtime](#)[└ **API Files**](#)[└ API Collections](#)[└ API Settings](#)[└ API Logs](#)[└ API Crons](#)[└ API Backups](#)[└ API Health](#)[Extend with
Go](#)[Extend with
JavaScript](#)[JS Overview](#)[JS Event hooks](#)[JS Routing](#)[JS Database](#)[JS Record operations](#)[JS Collection operations](#)[JS Migrations](#)[JS Jobs scheduling](#)[JS Sending emails](#)[JS Rendering templates](#)

Web APIs reference - API Files

Files are uploaded, updated or deleted via the [Records API](#).

The File API is usually used to fetch/download a file resource (with support for basic image manipulations, like generating thumbs).

[Download / Fetch file](#)[Generate protected file token](#)[← Prev: API Realtime](#)[Next: API Collections →](#)

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation





Introduction

Going to production

Web APIs reference

API Records

API Realtime

API Files

API Collections

API Settings

API Logs

API Crons

API Backups

API Health

Extend with
Go

Extend with
JavaScript

JS Overview

JS Event hooks

JS Routing

JS Database

JS Record operations

JS Collection operations

JS Migrations

JS Jobs scheduling

JS Sending emails

JS Rendering templates

Web APIs reference - API Settings

List settings



Update settings



Test S3 storage connection



Send test email



Generate Apple client secret



← Prev: API Collections

Next: API Logs →

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation





Introduction

Going to production

Web APIs reference

API Records

API Realtime

API Files

API Collections

API Settings

API Logs

API Crons

API Backups

API Health

Extend with
Go

Extend with
JavaScript

JS Overview

JS Event hooks

JS Routing

JS Database

JS Record operations

JS Collection operations

JS Migrations

JS Jobs scheduling

JS Sending emails

JS Rendering templates

Web APIs reference - API Collections

List collections



View collection



Create collection



Update collection



Delete collection



Truncate collection



Import collections



Scaffolds



← Prev: API Files

Next: API Settings →

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation





Introduction

Going to production

Web APIs reference

API Records

API Realtime

API Files

API Collections

API Settings

API Logs

API Crons

API Backups

API Health

Extend with
Go

Extend with
JavaScript

JS Overview

JS Event hooks

JS Routing

JS Database

JS Record operations

JS Collection operations

JS Migrations

JS Jobs scheduling

JS Sending emails

JS Rendering templates

Web APIs reference - API Logs

List logs



View log



Logs statistics



← Prev: API Settings

Next: API Crons →

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation





Introduction

Going to production

Web APIs reference

API Records

API Realtime

API Files

API Collections

API Settings

API Logs

API Crons

API Backups

API Health

Extend with
Go

Extend with
JavaScript

JS Overview

JS Event hooks

JS Routing

JS Database

JS Record operations

JS Collection operations

JS Migrations

JS Jobs scheduling

JS Sending emails

JS Rendering templates

Web APIs reference - API Crons

List cron jobs



Run cron job



← Prev: API Logs

Next: API Backups →

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation





Introduction

Going to production

Web APIs reference

API Records

API Realtime

API Files

API Collections

API Settings

API Logs

API Crons

API Backups

API Health

Extend with
Go

Extend with
JavaScript

JS Overview

JS Event hooks

JS Routing

JS Database

JS Record operations

JS Collection operations

JS Migrations

JS Jobs scheduling

JS Sending emails

JS Rendering templates

Web APIs reference - API Health

Health check



← Prev: API Backups

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation





Introduction

Going to production

Web APIs reference

API Records

API Realtime

API Files

API Collections

API Settings

API Logs

API Crons

API Backups

API Health

Extend with
Go

Extend with
JavaScript

JS Overview

JS Event hooks

JS Routing

JS Database

JS Record operations

JS Collection operations

JS Migrations

JS Jobs scheduling

JS Sending emails

JS Rendering templates

Web APIs reference - API Backups

List backups



Create backup



Upload backup



Delete backup



Restore backup



Download backup



← Prev: API Crons

Next: API Health →

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation

